

L'ÉPOPÉE DES SILENCES

Spécifications Fonctionnelles Détaillées — SFD v2.0.0

RPG roguelite narratif full web — Palais mental — Runs procédurales — Backend serveur-autoritaire

Métadonnée	Valeur
Projet	L'épopée des silences
Ancien nom	RPG_ESI07
Document	SFD — Spécifications Fonctionnelles Détaillées
Version cible	app-2.0.0
Client principal	Web — Vue 3 / TypeScript
Client Unity	Legacy v1 / hors cible principale v2
Date	2026-05-30
Statut	Cadrage fonctionnel pour développement v2

Résumé exécutif

Ce document formalise la version 2.0.0 comme une refonte full web : le jeu devient un RPG roguelite narratif dans lequel le joueur explore son propre Palais mental. Les runs sont générées par seed, les choix sont visibles mais irréversibles, les Lois du Palais modifient la génération, les événements, le combat, la narration et les récompenses, et le backend reste source de vérité.

Sommaire

- 1. Objectif et périmètre
- 2. Vision produit et univers
- 3. Glossaire fonctionnel
- 4. Modèle fonctionnel du Palais mental
- 5. Définition d'une run
- 6. Génération des pièces, noeuds et choix
- 7. Moteur des Lois du Palais
- 8. Événements de carte
- 9. Progression, récompenses et objets
- 10. Compagnons et équipe
- 11. Combat - cadrage fonctionnel macro
- 12. Narration modulaire, Elise et Tome
- 13. Leaderboard, seeds et saisons
- 14. Parcours utilisateur
- 15. Wireframes et écrans cible
- 16. Exigences fonctionnelles détaillées
- 17. Règles serveur-autoritaire
- 18. Architecture technique cible
- 19. Event Sourcing et événements métier
- 20. Modèles conceptuels de données
- 21. API fonctionnelles attendues
- 22. Administration et contenu
- 23. Exigences non fonctionnelles
- 24. Roadmap de réalisation v2
- 25. Critères d'acceptation et backlog initial
- 26. Risques, hypothèses et points ouverts

1. Objectif et périmètre

Les SFD définissent le comportement attendu de la version 2.0.0. Elles servent de référence pour prioriser le développement, découper les services, concevoir les écrans, identifier les données et stabiliser la logique serveur-autoritaire.

Inclus	Reporté / document futur
Vision gameplay full web	Scénarii narratifs complets
Fonctionnement du Palais mental	Équilibrage final des combats
Run, seed, Markov, événements	Direction artistique finale
Lois du Palais et impacts systémiques	Liste exhaustive des objets et ennemis
Progression temporaire/permanente	Formules définitives de dégâts
Architecture technique cible	Implémentation détaillée du code

Décision structurante

La v2 abandonne Unity comme client principal. Le client principal devient une application web. Unity reste une trace historique de la v1 et peut être conservé comme client legacy ou expérimentation, mais ne pilote plus la roadmap v2.

2. Vision produit et univers

L'épopée des silences est un RPG roguelite narratif jouable depuis navigateur. Le joueur explore son propre Palais mental, projection de son inconscient et de son subconscient. Les ennemis représentent des traumatismes, les événements représentent des souvenirs ou manifestations mentales, et les boss incarnent des traumatismes inachevés.

Le Tome des silences est un livre physique dans l'univers du jeu. Le tome originel décrit le Palais et s'enrichit continuellement de nouvelles pièces, lois, environnements, ennemis et fragments. Chaque run écrit progressivement le Tome du joueur : son parcours, ses choix, ses échecs et ses victoires deviennent un récit.

Promesse produit

Chaque run est à la fois une tentative de jeu, une exploration mentale et un chapitre écrit dans le Tome du joueur.

2.1 Piliers de conception

Pilier	Description
Full web	Le jeu est accessible depuis navigateur, sans build Unity ni téléchargement obligatoire.
Roguelite narratif	Chaque run est unique, mais contribue à une progression permanente et à une narration globale.
Palais mental	La génération procédurale est justifiée par la nature instable et personnelle du Palais.
Serveur-autoritaire	Le client web envoie des intentions ; le backend décide des résultats.
Event Sourcing	L'histoire d'une run est reconstruite à partir d'événements immuables.

3. Glossaire fonctionnel

Terme	Définition
-------	------------

Tome des silences	Livre qui archive et écrit les traversées du Palais.
Palais mental	Projection de l'inconscient du joueur.
Run	Exploration temporaire d'un Palais, générée par seed.
Pièce / carte	Espace mental contenant des choix ou événements.
Noeud	Choix visible et cliquable menant à un événement.
Événement	Résolution d'un noeud : combat, mémoire, PNJ, repos, objet, loi, etc.
Loi du Palais	Règle systémique modifiant plusieurs dimensions de la run.
Him'Lit	Boss final récurrent, influencé par le parcours de run.
Elise	Guide / matrice narrative accompagnant le joueur.
Seed	Identifiant permettant de reproduire une structure de run.
Fragment narratif	Élément de récit conditionnel injecté dans un événement ou dialogue.
Compagnon	Allié récupéré via événement, contrôlé par le joueur.

4. Modèle fonctionnel du Palais mental

SFD V2 — Modèle fonctionnel du Palais mental

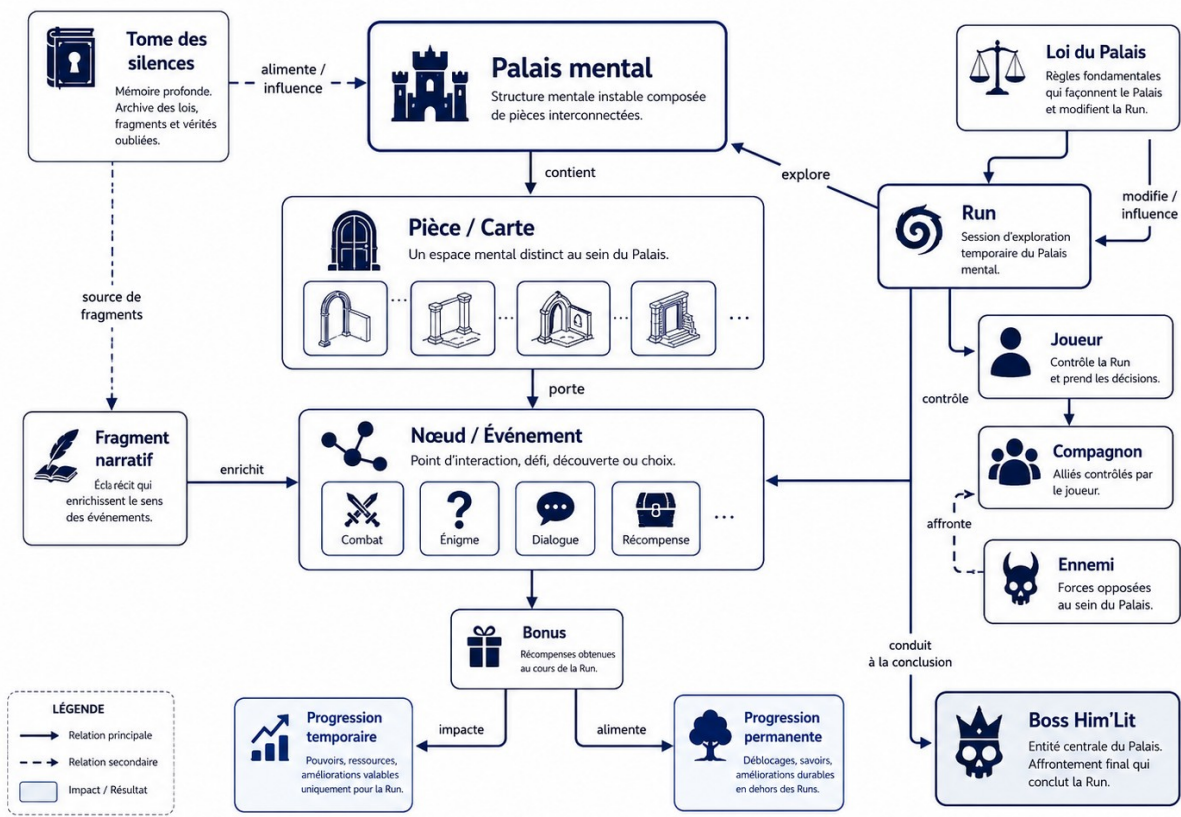


Figure 1 — Modèle fonctionnel du Palais mental

Concept	Rôle système	Impact gameplay
Palais mental	Conteneur principal des runs	Donne le cadre, les lois et la logique des pièces
Run	Instance temporaire d'exploration	Définit la progression, les choix,

		l'historique
Pièce	Unité de progression	Expose un ensemble de noeuds/événements
Noeud	Choix joueur	Déclenche un événement et verrouille le retour arrière
Loi	Modificateur transversal	Altère plusieurs sous-systèmes
Fragment narratif	Contenu conditionnel	Donne sens aux événements et au Tome
Bonus	Récompense ou altération	Influence la run ou la progression permanente

5. Définition d'une run

Une run correspond à une exploration temporaire du Palais mental, générée à partir d'une seed et guidée par une succession de choix irréversibles. Elle commence au seuil du Palais, propose quatre choix initiaux, puis se développe en embranchements successifs jusqu'à Him'Lit.

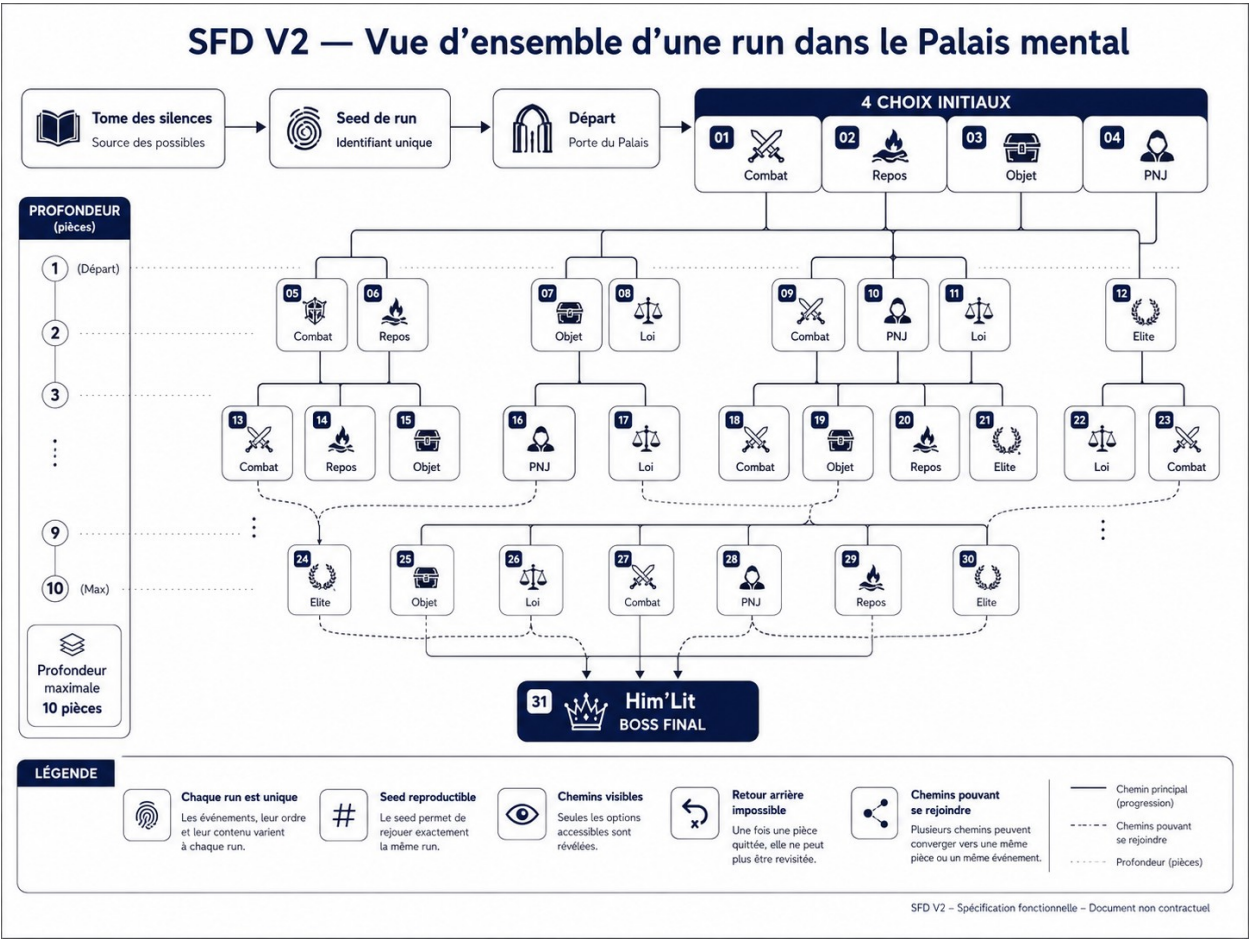


Figure 2 — Vue d'ensemble d'une run dans le Palais mental

5.1 Règles de structure

Règle	Description
Départ	Un point d'entrée unique dans le Palais.

Choix initiaux	Quatre choix visibles dès le lancement de la run.
Choix suivants	Chaque choix peut mener à 1 à 4 choix suivants.
Profondeur maximale	10 pièces maximum avant la confrontation finale.
Choix par pièce	Jusqu'à 70 choix théoriques par pièce pour laisser de la place à la génération et aux combinaisons.
Retour arrière	Interdit après validation du choix.
Rejonction	Certains chemins peuvent converger vers une même pièce ou un même événement.
Visibilité	Les prochains noeuds visibles sont affichés au joueur.
Boss final	Him'Lit conclut la run.

5.2 États de run

État	Déclencheur	Comportement
Created	Création de la run	Aucune action joueur résolue
Active	Démarrage confirmé	Le joueur peut choisir des noeuds
RoomResolved	Événement résolu	Le joueur choisit une sortie
BossReached	Dernière pièce atteinte	Him'Lit est généré
Completed	Him'Lit vaincu	Score et récompenses finalisés
Failed	Défaite du joueur	Fin de run et projection de résultat
Abandoned	Abandon volontaire	Run close avec statut abandon

6. Génération des pièces, noeuds et choix

La génération doit produire un résultat varié mais contrôlé. Elle repose sur une seed, une version de générateur, une matrice de Markov et les Lois du Palais actives. Le système doit pouvoir évoluer sans casser les seeds existantes : chaque run doit stocker la version du générateur utilisée.

```

StartRun(playerId)
-> create runId
-> generate seed
-> load generatorVersion + markovMatrixVersion
-> select initial Palace Laws
-> generate first room
-> generate 4 initial nodes
-> persist RunStarted + RoomGenerated events

```

6.1 Paramètres de génération

Paramètre	Description	Persisté ?
seed	Identifiant reproductible de génération	Oui
generatorVersion	Version du générateur	Oui
markovMatrixVersion	Version de la matrice	Oui
initialLaws	Lois initiales	Oui
playerSnapshot	État de départ du joueur	Oui
depth	Profondeur actuelle	Oui
history	Actions et choix précédents	Oui

6.2 Matrice de Markov

La matrice de Markov pondère les transitions entre environnements, types de pièces et événements. Elle ne remplace pas les règles métier : elle guide la génération, tandis que le backend valide les contraintes de cohérence.

État courant	Transition privilégiée	Exemple de conséquence
Forêt	Nature / Mémoire / Combat	Him'Lit peut obtenir des compétences de type forêt
Mémoire	Dialogue / Fragment / PNJ	Elise intervient plus souvent
Rupture	Malédiction / Elite	Risque accru, meilleure récompense
Repos	Objet / Marchand	Rééquilibrage après combat difficile
Elite	Récompense rare / Loi	Accès à une altération forte du Palais

7. Moteur des Lois du Palais

Les Lois du Palais sont un système central, modulable et extensible. Une loi peut modifier simultanément plusieurs sous-systèmes : génération, événements, combat, récompenses, narration et Him'Lit. Elle doit être représentée comme une règle branchable, versionnée et testable.

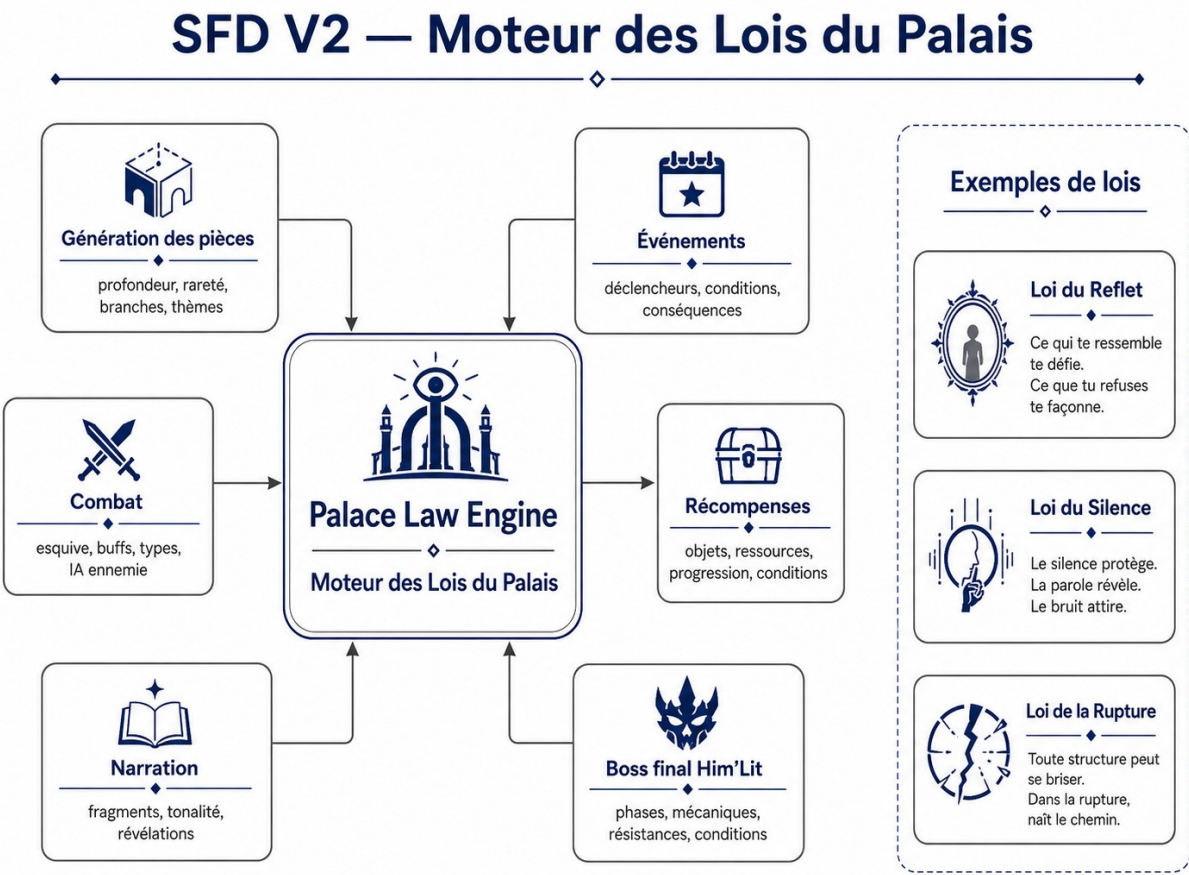


Figure 3 — Moteur des Lois du Palais

Domaine impacté	Exemple d'impact
Génération	Modifier probabilité des types de pièces, profondeur, rareté, rejonctions
Événements	Masquer certains choix, ajouter des conditions, modifier les conséquences
Combat	Modifier esquive, buffs, types, résistances, IA ennemie
Récompenses	Augmenter rareté, changer conditions d'obtention, imposer

	un coût
Narration	Modifier le ton, les fragments, les révélations et les dialogues Elise
Him'Lit	Influencer compétences, phases, résistances et mécaniques finales

7.1 Exigences du moteur de lois

ID	Exigence
LAW-001	Une loi doit être activable, désactivable et versionnée.
LAW-002	Une loi peut cibler plusieurs domaines d'impact.
LAW-003	Une loi doit pouvoir être ajoutée sans modifier le coeur du moteur.
LAW-004	Les impacts d'une loi doivent être traçables dans l'historique de run.
LAW-005	Les effets d'une loi doivent être visibles ou partiellement visibles selon la loi.
LAW-006	Him'Lit doit tenir compte des lois dominantes traversées pendant la run.

8. Événements de carte

Chaque noeud visible correspond à un événement ou à une promesse d'événement. Le système doit rester extensible afin d'ajouter régulièrement de nouveaux types d'événements et variantes narratives.

Type	But	Résolution typique	Événement émis
Combat	Affrontement	Démarre un combat puis récompense	CombatStarted / CombatEnded
Mini-boss / Elite	Défi supérieur	Combat dur, récompense rare	EliteEncounterStarted
Objet	Ressource ou équipement	Ajout temporaire ou permanent	RewardGranted
PNJ	Dialogue / choix	Fragment, compagnon, quête, malus	NarrativeFragmentRevealed
Mémoire	Révélation	Page du Tome, dialogue Elise	TomePageUnlocked
Repos	Récupération	Soin, PP, stabilisation	RestResolved
Marchand	Échange	Achat, vente, amélioration	TradeResolved
Loi	Altération du Palais	Ajout/modification de loi	PalaceLawApplied
Malédiction	Risque fort	Coût immédiat, récompense élevée	CurseAccepted
Rare	Événement exceptionnel	Compagnon, relique, branche spéciale	RareEventResolved

9. Progression, récompenses et objets

La v2 distingue la progression temporaire de run et la progression permanente du joueur. Les objets temporaires favorisent l'adaptation tactique pendant la run. Les objets permanents et reliques renforcent la boucle de rétention entre runs.

Catégorie	Durée	Exemple
Objet temporaire	Run uniquement	Potion, buff, clé mentale
Objet permanent	Entre runs	Relique, artefact, équipement durable

Amélioration temporaire	Run uniquement	+10% esquive, PP récupérés
Amélioration permanente	Compte joueur	Compagnon débloqué, page du Tome, bonus de départ
Ressource de run	Run uniquement	Fragments, énergie, monnaie temporaire
Ressource permanente	Compte joueur	Monnaie rare, connaissance, relique

10. Compagnons et équipe

Le joueur combat seul au début. Neige est le premier compagnon disponible. Les compagnons sont récupérés via des événements de carte, pas capturés directement en combat. À terme, le joueur peut combattre avec jusqu'à quatre compagnons actifs.

Règle	Décision
Compagnon initial	Neige
Obtention	Événements de carte ou narration
Capture en combat	Non
Compagnons actifs max	4 compagnons + joueur
Contrôle	Le joueur contrôle les compétences des compagnons
Enclos	Zone du site affichant les compagnons débloqués
Progression compagnon	À définir dans un document dédié, mais supportée par le modèle

11. Combat - cadrage fonctionnel macro

Le combat v2 doit conserver la profondeur stratégique d'un système à quatre compétences fixes et ressources limitées, tout en ajoutant du dynamisme. La structure exacte fera l'objet d'un document combat, mais le cadre macro est fixé.

Élément	Décision v2
Format maximal	5 alliés contre 6 ennemis
Alliés	Joueur + jusqu'à 4 compagnons
Contrôle	Manuel pour joueur et compagnons
Compétences	4 compétences actives par combattant
Ressources	PP / charges + mana selon compétence
Types	Table de types propre au jeu
Affinités	Super efficace / faible / résisté
États altérés	Oui
Esquive	Oui, mécanique dynamique envisagée
Tour par tour / ATB	Base tour par tour, évolution ATB possible
Backend	Résolution autoritaire côté serveur

Principe de dynamisme

Le combat doit rester lisible, mais éviter la monotonie. Les mécaniques dynamiques possibles sont : esquive, initiative, contre-effets, synergies de compagnons, actions bonus et effets liés aux Lois du Palais.

12. Narration modulaire, Elise et Tome

Le scénario sera écrit ultérieurement sous forme de scénarii et fragments. Les SFD doivent néanmoins prévoir le système qui accueillera ces contenus : fragments narratifs conditionnels, dialogues Elise, pages du Tome, événements liés aux types de pièces et aux ennemis présents.

Système	Fonction
Elise	Guide le joueur, commente, révèle ou questionne les événements
Fragments narratifs	Textes courts déclenchés par conditions
Pages du Tome	Récompenses narratives et traces permanentes
Scénarii de run	Arcs modulaires recomposés selon le parcours
Dialogues post-combat	Révèlent un traumatisme, une absence ou une vérité
Boss d'étage / final	Orienté le thème narratif dominant

Exemple de déclenchement narratif :

Si roomTheme = "mort" et bossTheme = "faucheuse" et playerHistory contient "perte"

-> afficher fragment : "[NOM] me manque énormément."

-> EliseDialogueResolver sélectionne une réponse adaptée

-> TomeWriter écrit une trace narrative dans le Tome du joueur

13. Leaderboard, seeds et saisons

Le leaderboard sert à la compétition et à la valorisation des Tomes écrits par les joueurs. Les seeds peuvent être affichées pour permettre la comparaison, le partage et la rejouabilité.

Élément	Description
Score	Calculé selon profondeur, boss, difficulté, risques, récompenses et temps
Seed	Affichable et partageable
Tome du joueur	Nom ou trace narrative liée à la run
Saison	Période de classement avec récompenses
Récompenses saisonnières	Objets, reliques, cosmétiques ou pages spéciales
Rejouabilité	Les joueurs peuvent tenter une seed partagée

14. Parcours utilisateur

14.1 Parcours nominal : run complète

1. Connexion utilisateur
2. Ouverture du Tome des silences
3. Démarrage d'une run
4. Génération seed + lois initiales
5. Affichage des 4 choix initiaux
6. Sélection d'un noeud
7. Résolution de l'événement
8. Mise à jour de l'état de run
9. Choix suivant
10. Confrontation à Him'Lit

11. Fin de run et projection leaderboard

14.2 Parcours événement de carte

- 12. Le joueur consulte les choix disponibles
- 13. Il ouvre un aperçu du noeud
- 14. Il confirme son choix
- 15. Le backend valide que le noeud est accessible
- 16. Le backend résout l'événement
- 17. Le frontend affiche résultat, dialogue, combat ou récompense
- 18. Le backend écrit les événements de run

14.3 Parcours récompense

- 19. Le backend propose une ou plusieurs récompenses
- 20. Le joueur sélectionne une récompense
- 21. Le backend vérifie la disponibilité
- 22. L'effet est appliqué à la run ou au profil permanent
- 23. Un événement RewardSelected est écrit

15. Wireframes et écrans cible

Les écrans ci-dessous ne représentent pas la direction artistique finale. Ils servent à stabiliser la logique fonctionnelle : carte de progression, choix disponibles, informations de run, lois actives, équipe, bonus, récompenses et actions souris.

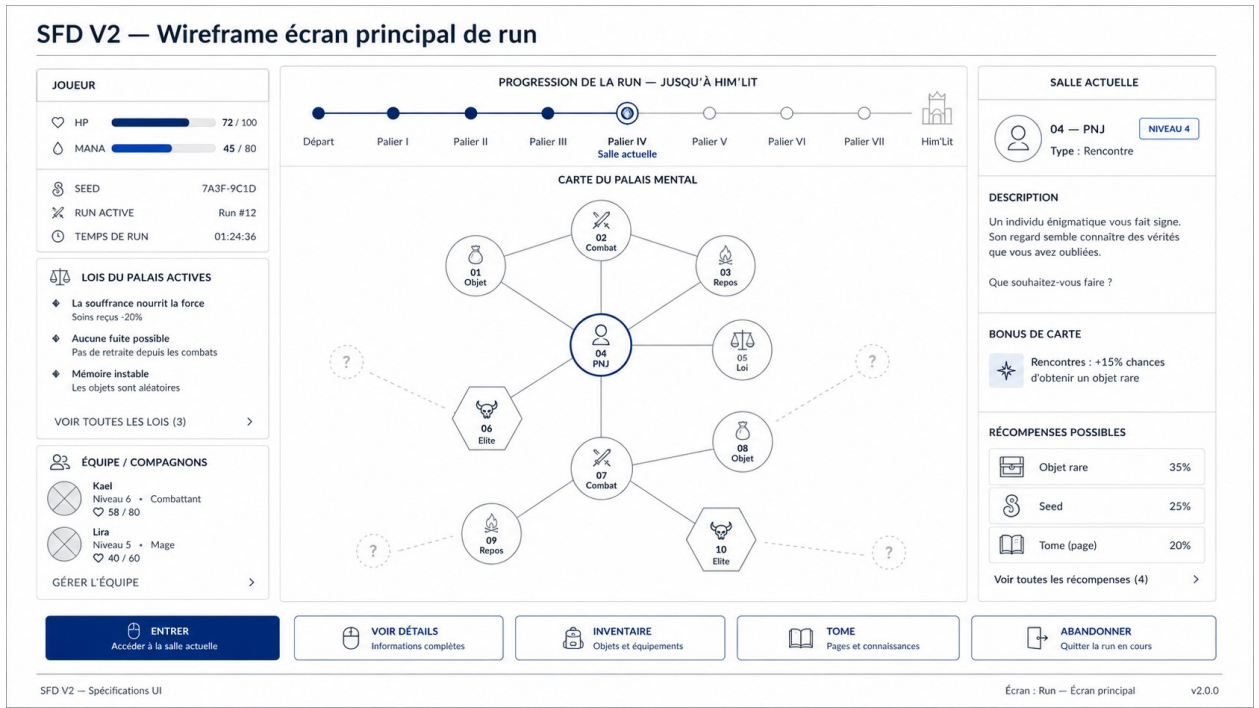


Figure 4 — Wireframe écran principal de run

SFD V2 — Wireframe sélection de salle / événements

Vous avez terminé la salle. Choisissez votre prochain chemin.

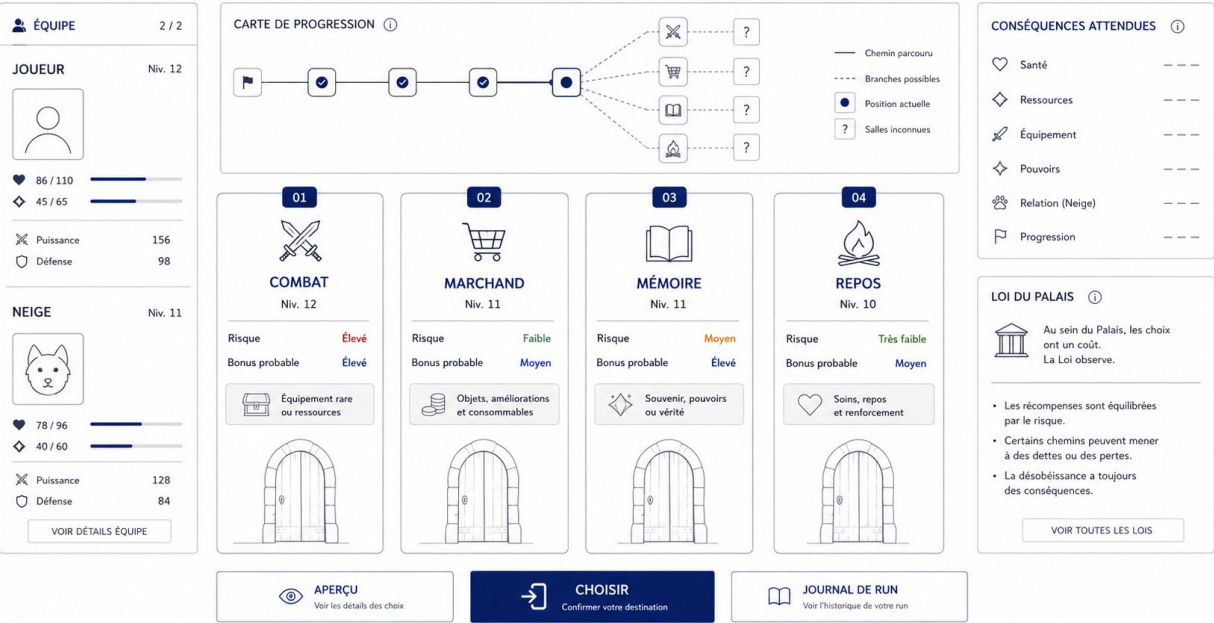


Figure 5 — Wireframe sélection de salle / événements

Écran	Fonction	Composants clés
Écran principal de run	Afficher la salle actuelle et la carte du Palais	Progression, noeuds, lois, équipe, détails de salle
Sélection de salle	Choisir le prochain embranchement	4 cartes/portes, risque, bonus probable, aperçu
Inventaire	Gérer objets temporaires et permanents	Liste objets, effets, rareté, durée
Tome	Consulter pages et fragments débloqués	Pages, fragments, historique de run
Enclos	Voir compagnons débloqués	Neige, compagnons futurs, stats, compétences
Leaderboard	Comparer les runs	Score, seed, saison, Tome, statut

16. Exigences fonctionnelles détaillées

ID	Exigence	Priorité
RUN-001	Le joueur peut démarrer une nouvelle run depuis le client web.	Must
RUN-002	Une seed unique est générée et stockée pour chaque run.	Must
RUN-003	La run démarre avec 4 choix initiaux visibles.	Must
RUN-004	Chaque noeud peut ouvrir 1 à 4 choix suivants.	Must
RUN-005	Le retour arrière est impossible après validation.	Must
RUN-006	La profondeur maximale d'une run est de 10 pièces.	Must
RUN-007	Les chemins peuvent se rejoindre.	Should
RUN-008	Le joueur peut abandonner une run.	Should

GEN-001	La génération utilise seed + version générateur + matrice Markov.	Must
GEN-002	Les lois actives influencent la génération.	Must
EVT-001	Les types d'événements sont extensibles.	Must
LAW-001	Une loi peut impacter génération, événements, combat, récompenses, narration et boss.	Must
CBT-001	Le backend résout les combats.	Must
CMP-001	Le joueur peut obtenir des compagnons par événements.	Should
NAR-001	Le système supporte des fragments narratifs conditionnels.	Must
LDB-001	Le leaderboard affiche score, seed et statut de run.	Should

17. Règles serveur-autoritaire

Le frontend ne transmet que des intentions. Il ne peut pas déclarer un résultat définitif : il ne décide pas des dégâts, récompenses, loot, score, progression, seed, statut de run ou leaderboard.

Intention frontend	Validation backend	Résultat backend
StartRun	Utilisateur authentifié, pas de run active incompatible	RunStarted + seed + choix initiaux
ChooseNode	Noeud accessible, run active, pas déjà résolu	NodeSelected + EventResolved potentiel
ChooseCombatAction	Tour valide, compétence disponible, PP suffisants, cible valide	CombatActionResolved
SelectReward	Récompense proposée et non consommée	RewardSelected + effets appliqués
AbandonRun	Run active	RunAbandoned + projections

Anti-triche fonctionnel

Le modèle full web ne rend pas le client fiable. Toute donnée critique doit être contrôlée par le backend : génération, combat, inventaire, progression, récompense, score et leaderboard.

18. Architecture technique cible

L'architecture cible n'est pas une micro-architecture fragmentée à l'extrême. Le coeur gameplay est regroupé dans un Game Engine Service fort afin de conserver la cohérence entre run, génération, événements, combat, récompenses et narration. Les domaines périphériques sont isolés en services spécialisés.

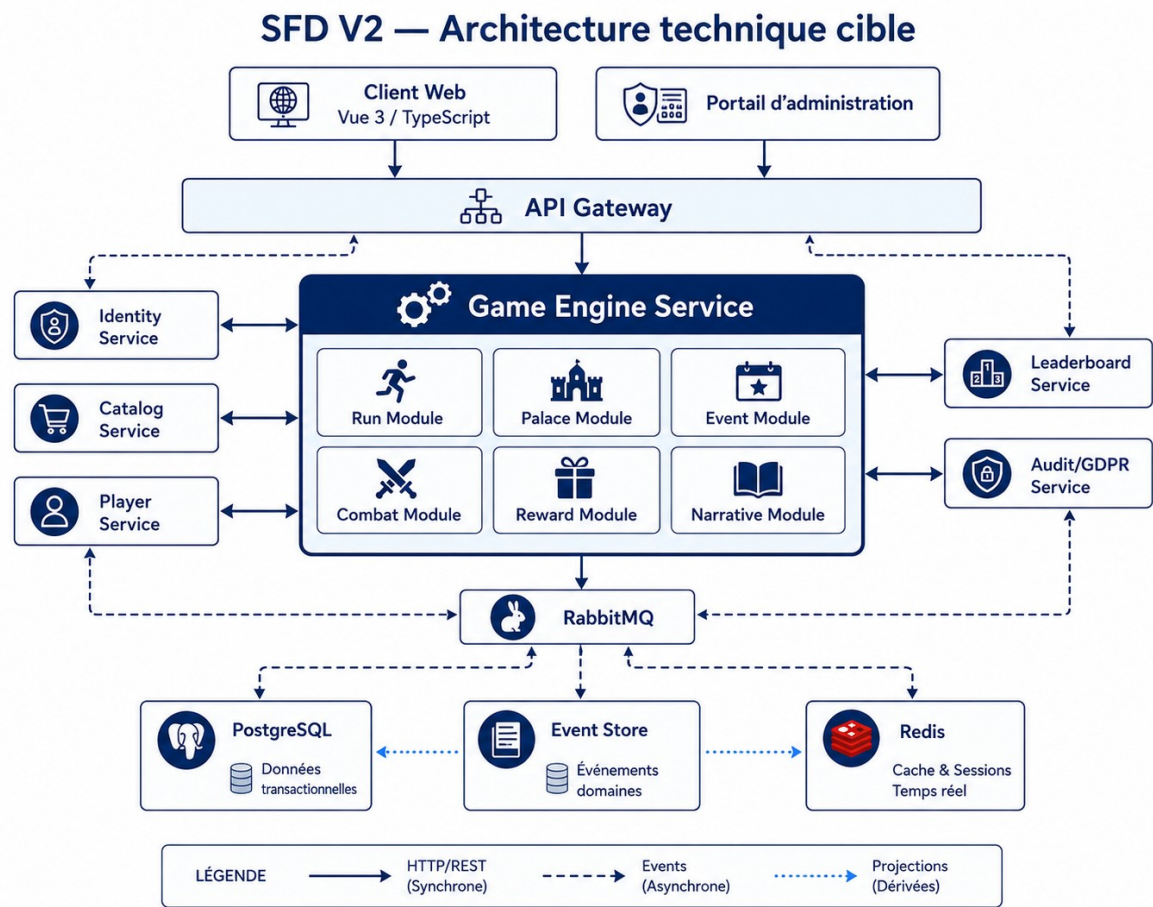


Figure 6 — Architecture technique cible

Composant	Responsabilité
Client Web Vue 3	Interface de jeu, interactions souris, affichage des intentions et résultats
API Gateway	Point d'entrée, routing, sécurité, DTO publics, rate limiting
Game Engine Service	Run, Palais, lois, événements, combat, récompenses, narration
Identity Service	Comptes, JWT, MFA, rôles
Catalog Service	Référentiels : ennemis, objets, compétences, lois, événements
Player Service	Profil joueur, progression permanente
Leaderboard Service	Classements, seeds, saisons, projections
Audit/GDPR Service	Traçabilité, export, anonymisation, consentements
RabbitMQ	Communication événementielle
Event Store	Historique immuable des runs
PostgreSQL	Données transactionnelles
Redis	Cache, sessions et projections rapides

19. Event Sourcing et événements métier

La run est la meilleure candidate à l'Event Sourcing. L'état actuel d'une run peut être reconstruit à partir d'un journal append-only d'événements horodatés. Les read models alimentent ensuite les vues optimisées : état courant de run, profil joueur, leaderboard, audit et Tome.

SFD V2 – Cycle de vie d'une run et Event Sourcing

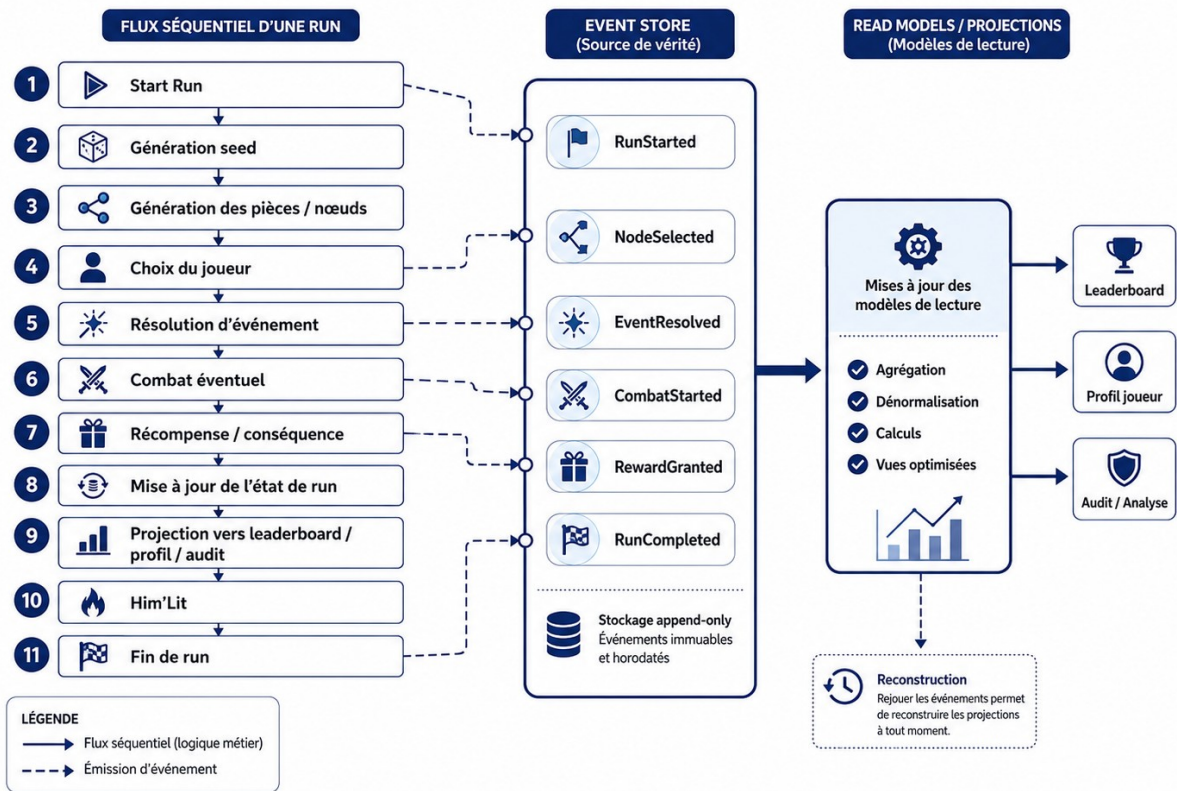


Figure 7 — Cycle de vie d'une run et Event Sourcing

Événement	Déclencheur	Projection impactée
RunStarted	Démarrage de run	RunState, PlayerHistory
RunSeedGenerated	Création seed	RunMetadata
RoomGenerated	Génération d'une pièce	CurrentRunMap
NodeSelected	Choix joueur	RunHistory
PalaceLawApplied	Application d'une loi	ActiveLaws
EventResolved	Résolution événement	RunState, Tome
CombatStarted	Début combat	CombatState
CombatActionResolved	Action de combat	CombatState
CombatEnded	Fin combat	RunState, Rewards
RewardOffered	Récompense proposée	RewardSelection
RewardSelected	Récompense choisie	Inventory, PlayerProfile
CompanionJoined	Compagnon obtenu	CompanionRoster
HimLitGenerated	Boss final généré	BossState
RunCompleted	Run gagnée	Leaderboard, Tome, PlayerStats
RunFailed	Run perdue	Leaderboard, PlayerStats
RunAbandoned	Abandon	PlayerHistory

20. Modèles conceptuels de données

Modèle	Attributs principaux	Owner
--------	----------------------	-------

Run	runId, playerId, seed, status, depth, currentRoomId, startedAt, endedAt	Game Engine
Room	roomId, runId, depth, mapType, level, lawContext	Game Engine
Node	nodeId, roomId, eventType, risk, rewardProfile, state	Game Engine
PalaceLaw	lawId, name, domains, priority, version, conditions	Catalog / Game Engine
EventTemplate	templateId, type, conditions, outcomes, narrativeTags	Catalog
NarrativeFragment	fragmentId, tags, speaker, text, triggers	Catalog / Narrative
Companion	companionId, name, skills, affinity, unlocked	Player / Catalog
Reward	rewardId, type, rarity, temporary, effects	Game Engine / Catalog
LeaderboardEntry	entryId, playerId, runId, score, seed, season	Leaderboard
TomePage	pageId, playerId, runId, content, unlockedAt	Player / Narrative

21. API fonctionnelles attendues

Les API ci-dessous sont indicatives. Elles décrivent les intentions fonctionnelles à exposer via la Gateway. Les contrats exacts seront définis dans les spécifications API.

Méthode	Route	Rôle
POST	/api/v2/runs	Démarrer une run
GET	/api/v2/runs/{runId}	Récupérer l'état courant
POST	/api/v2/runs/{runId}/nodes/{nodeId}/choose	Choisir un noeud
POST	/api/v2/runs/{runId}/events/{eventId}/choice	Choisir une option d'événement
POST	/api/v2/combat/{combatId}/actions	Envoyer une action de combat
POST	/api/v2/runs/{runId}/rewards/{rewardId}/select	Choisir une récompense
POST	/api/v2/runs/{runId}/abandon	Abandonner une run
GET	/api/v2/tome	Consulter le Tome du joueur
GET	/api/v2/companions	Lister les compagnons débloqués
GET	/api/v2/leaderboard	Consulter le classement

22. Administration et contenu

Le système doit permettre d'ajouter continuellement du contenu : types d'événements, ennemis, objets, compétences, lois, fragments narratifs et récompenses. L'administration est donc un enjeu fonctionnel majeur, même si l'interface complète peut être progressive.

Domaine administrable	Actions minimales
Enemies	Créer, modifier, désactiver, définir scaling et affinités
Items	Créer, modifier, définir rareté, durée, effets
Skills	Créer, définir PP, type, coût, effet
Palace Laws	Créer, versionner, activer, définir impacts
Event Templates	Créer des événements et leurs outcomes

Narrative Fragments	Créer fragments, tags, conditions, speakers
Rewards	Créer profils de récompense et probabilités
Seasons	Créer saison leaderboard, dates et récompenses

23. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Les actions courantes de run doivent répondre rapidement pour préserver le rythme de jeu.
Reproductibilité	Les runs doivent être restructuribles à partir de seed + événements.
Sécurité	Le backend est source de vérité pour toutes les données critiques.
Extensibilité	Les événements, lois et fragments narratifs doivent être ajoutables sans refonte.
Observabilité	Chaque run doit être traçable par correlationId et runId.
Testabilité	Les générateurs, lois, resolvers et projections doivent être testables unitairement.
Compatibilité	Les versions de générateur doivent être conservées pour éviter de casser les anciennes seeds.
Accessibilité	L'interface doit rester jouable principalement à la souris avec informations lisibles.

24. Roadmap de réalisation v2

Jalon	Objectif	Livrables
app-2.0.0-alpha.1	Cadrage complet	SFD, diagrammes, ADR, backlog
app-2.0.0-alpha.2	Socle technique	Repo, Gateway, Game Engine skeleton, Docker Compose
app-2.0.0-alpha.3	Génération de run	Seed, Markov, 4 choix initiaux, pièces
app-2.0.0-alpha.4	Prototype web	Écran run, choix de salle, état run
app-2.0.0-beta.1	Événements	Event resolver, récompenses, lois simples
app-2.0.0-beta.2	Combat minimal	Joueur + Neige, ennemis scalés, compétences
app-2.0.0-beta.3	Tome et narration	Elise, fragments, pages, historique
app-2.0.0-rc.1	Run complète	Him'Lit, leaderboard, tests, sécurité
app-2.0.0	Release stable	Version web jouable, publiée et documentée

25. Critères d'acceptation et backlog initial

25.1 Critères d'acceptation globaux

- Le jeu est jouable depuis navigateur.
- Le joueur peut effectuer une run complète jusqu'à Him'Lit.
- La génération est reproductible par seed et version de générateur.
- Les choix sont visibles, cliquables et irréversibles.
- Le backend valide toutes les actions critiques.
- Les événements de run sont persistés et exploitables en projections.

- Les Loïs du Palais modifient au moins trois domaines dans le MVP.
- Le leaderboard expose score, statut et seed.
- Le Tome du joueur reçoit au moins des traces de run.

25.2 Backlog initial

Epic	Objectif
EPIC-01 — Game Engine Foundation	Créer Run, Room, Node, Seed, State
EPIC-02 — Palace Generation	Markov, 4 choix initiaux, profondeur, rejonctions
EPIC-03 — Palace Laws	Moteur de lois, impacts et traces
EPIC-04 — Event Resolution	Types Combat, Objet, Repos, PNJ, Mémoire
EPIC-05 — Web Run UI	Écran principal et sélection de salle
EPIC-06 — Combat MVP	Combat simple backend-authoritative
EPIC-07 — Tome & Narrative	Elise, fragments, pages
EPIC-08 — Leaderboard	Score, seed, saison
EPIC-09 — Observability	CorrelationId, logs, audit
EPIC-10 — Administration	Gestion contenu de base

26. Risques, hypothèses et points ouverts

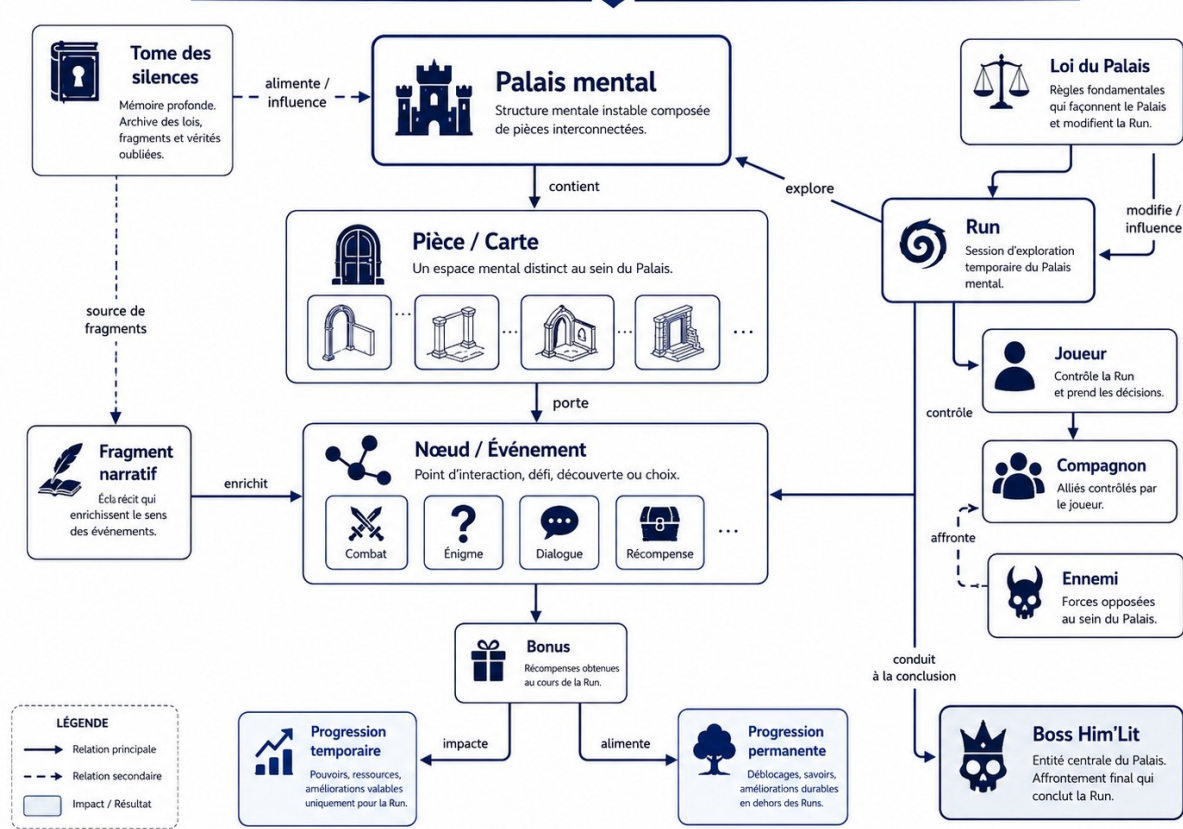
Sujet	Statut	Décision / risque
Formule de combat	Ouvert	À définir dans un document combat dédié
Table des types	Ouvert	Types propres au jeu, différents de Pokémon
Scénarii complets	Ouvert	Écriture ultérieure sous forme de scénarii et fragments
Lois définitives	Ouvert	Système extensible ; premières lois à prototyper
Système ATB	À arbitrer	Base tour par tour, ATB possible plus tard
70 choix par pièce	À surveiller	Risque UI et performance ; seuls choix accessibles visibles
Seed partageable	À confirmer	Intéressant pour leaderboard, mais nécessite compatibilité stricte
Compagnons multiples	Progressif	Neige d'abord, autres compagnons ensuite
Administration complète	Progressive	MVP d'administration avant interface complète

Conclusion de cadrage

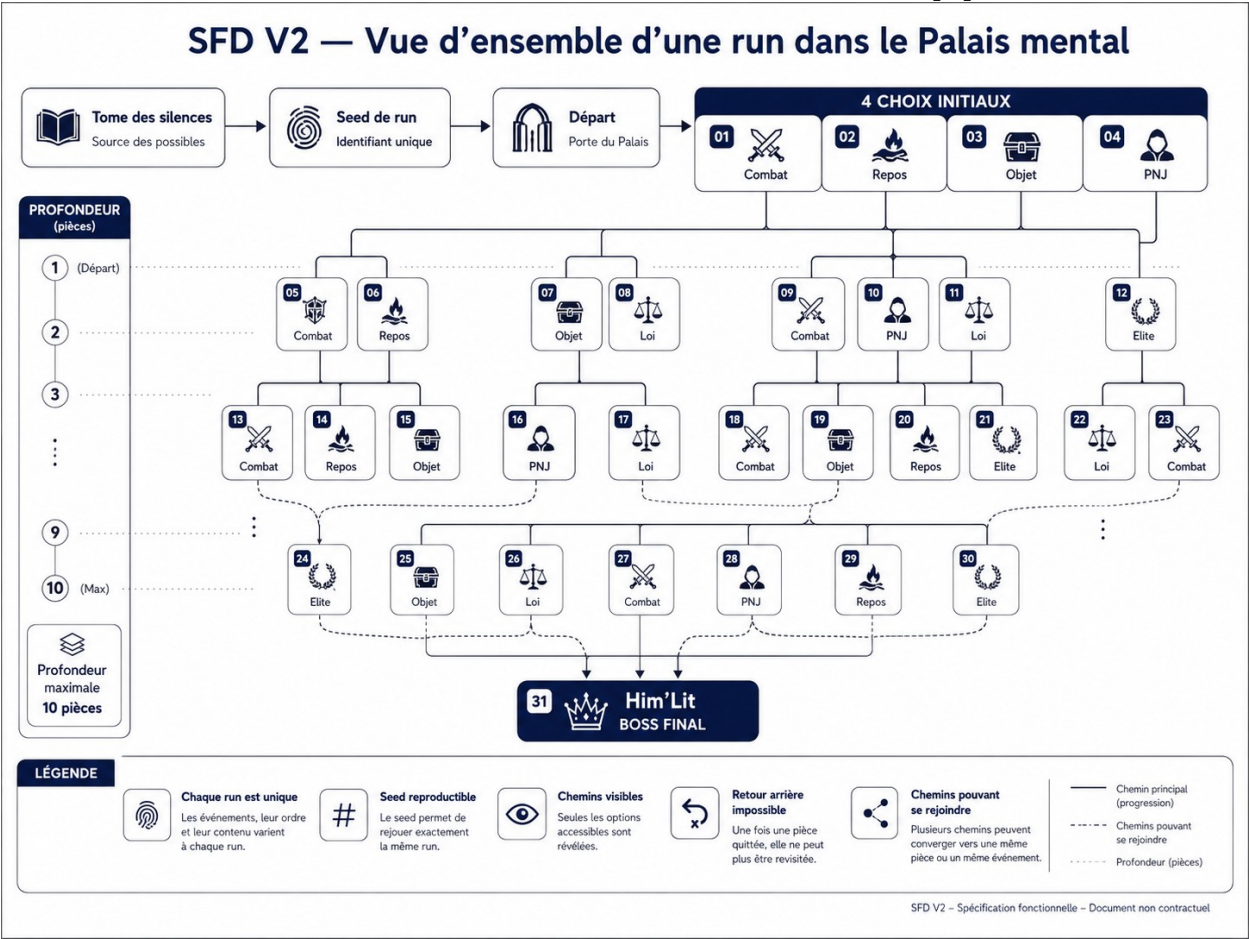
Les SFD fixent une base claire pour la V2 : full web, Palais mental, runs par seed, Lois modulaires, progression roguelite, narration par fragments, backend serveur-authoritaire et Event Sourcing centré sur la run. Le prochain document recommandé est le Game Design Document, puis les spécifications techniques API et événements.

Annexes visuelles

SFD V2 — Modèle fonctionnel du Palais mental

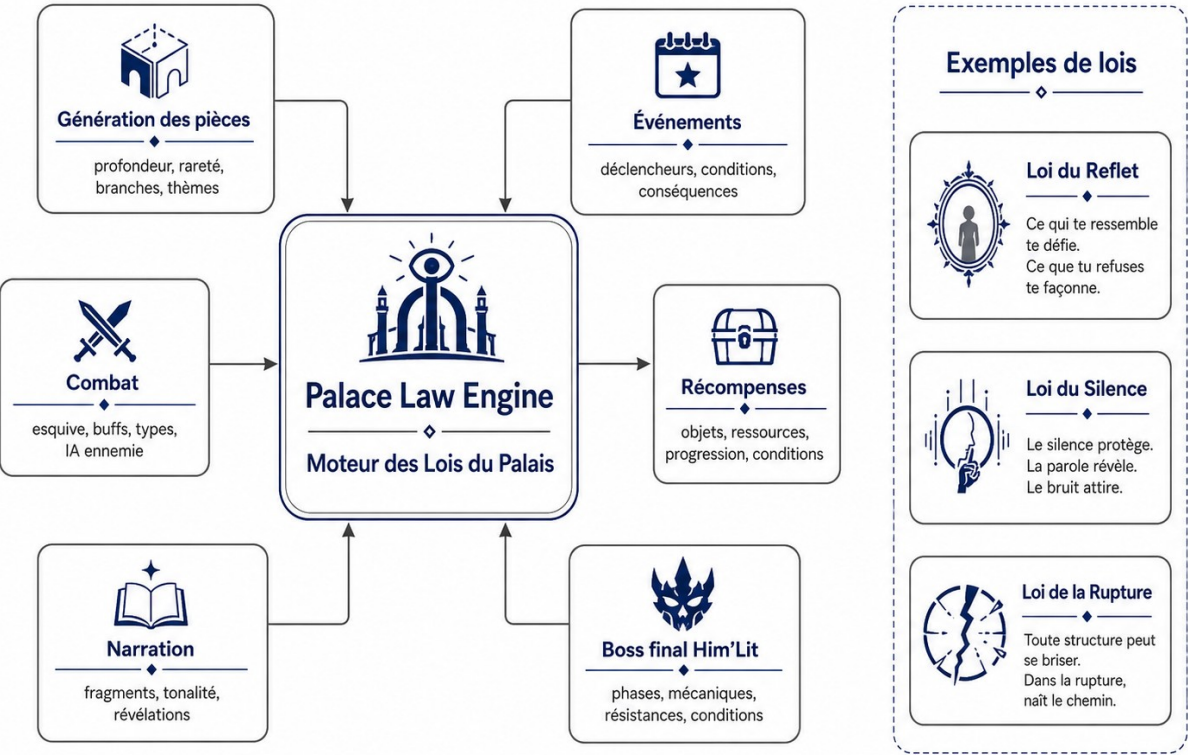


Annexe A — Modèle fonctionnel du Palais



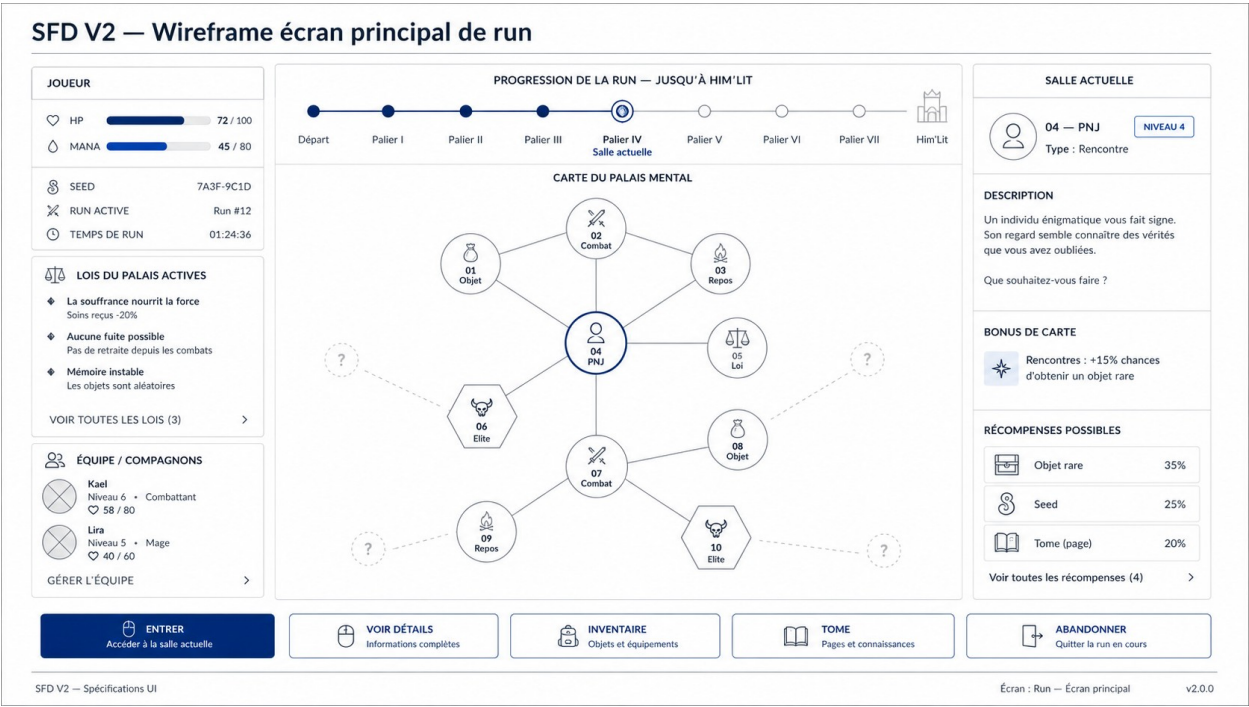
Annexe B — Vue d'ensemble de la run

SFD V2 — Moteur des Lois du Palais



Annexe C — Moteur des Lois du Palais

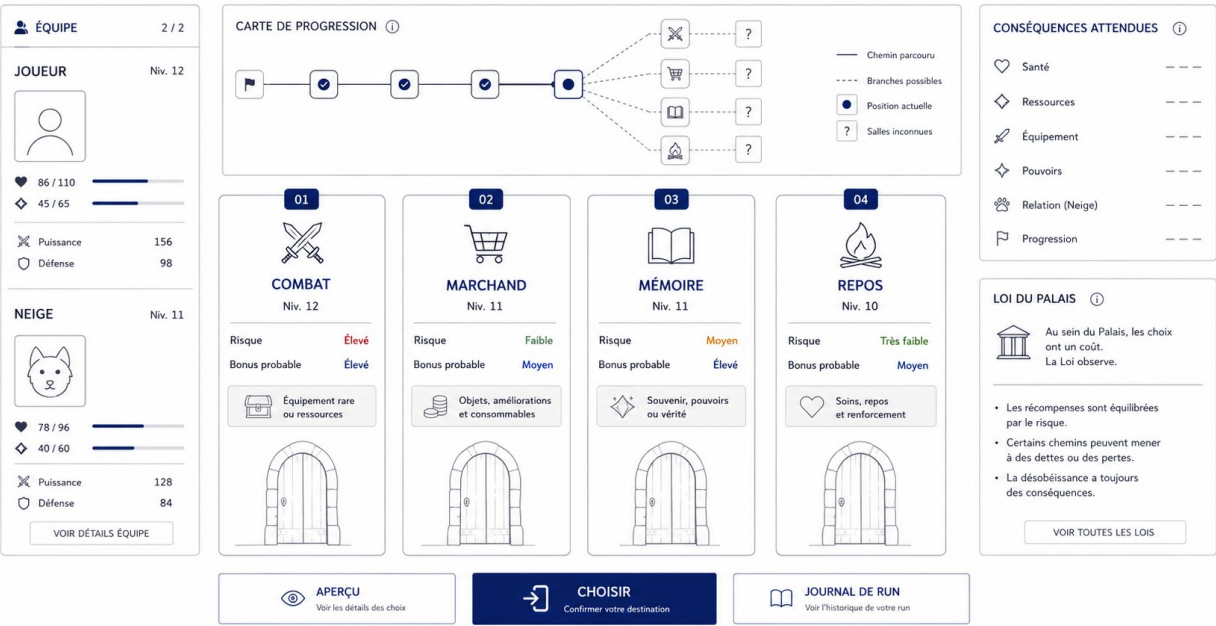
SFD V2 — Wireframe écran principal de run



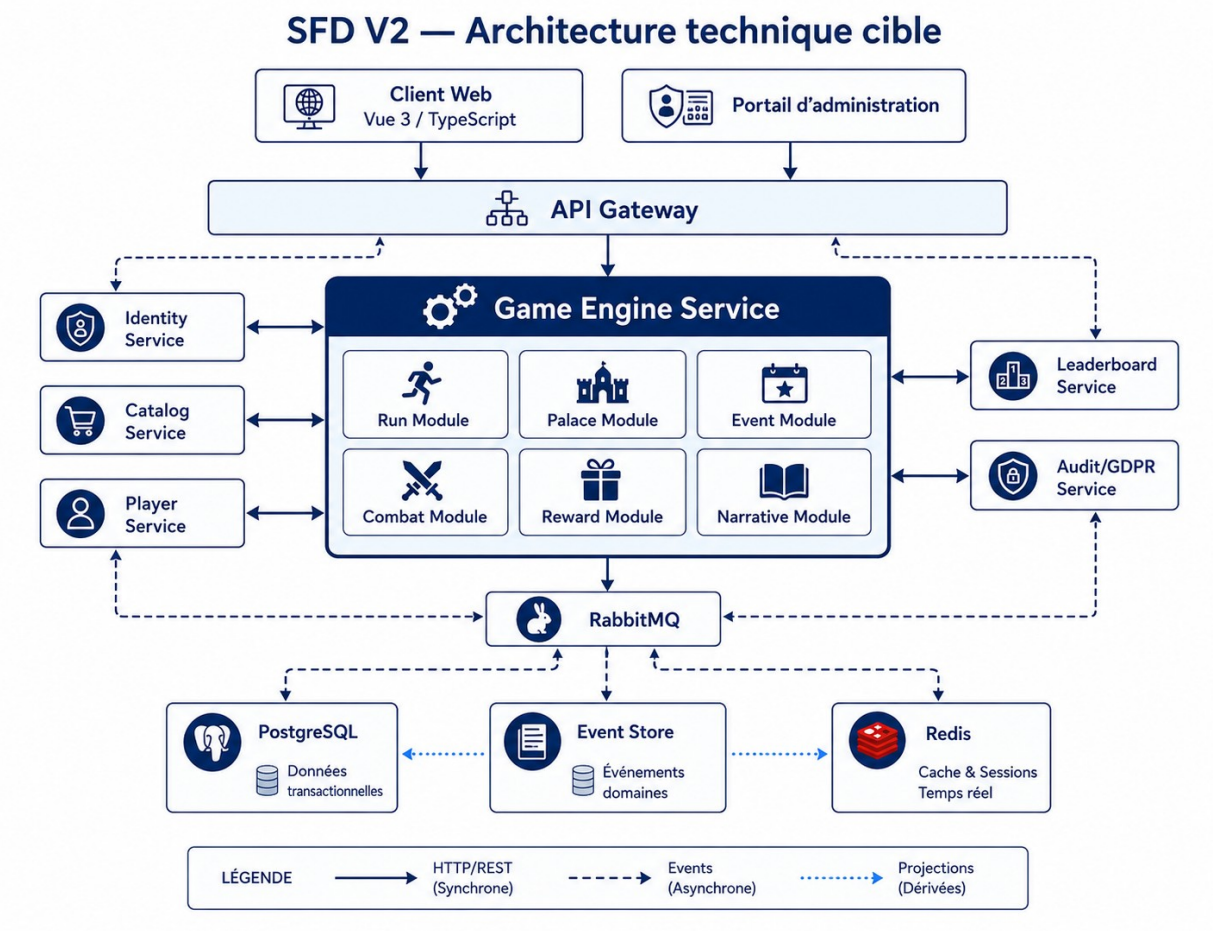
Annexe D — Écran principal de run

SFD V2 — Wireframe sélection de salle / événements

Vous avez terminé la salle. Choisissez votre prochain chemin.

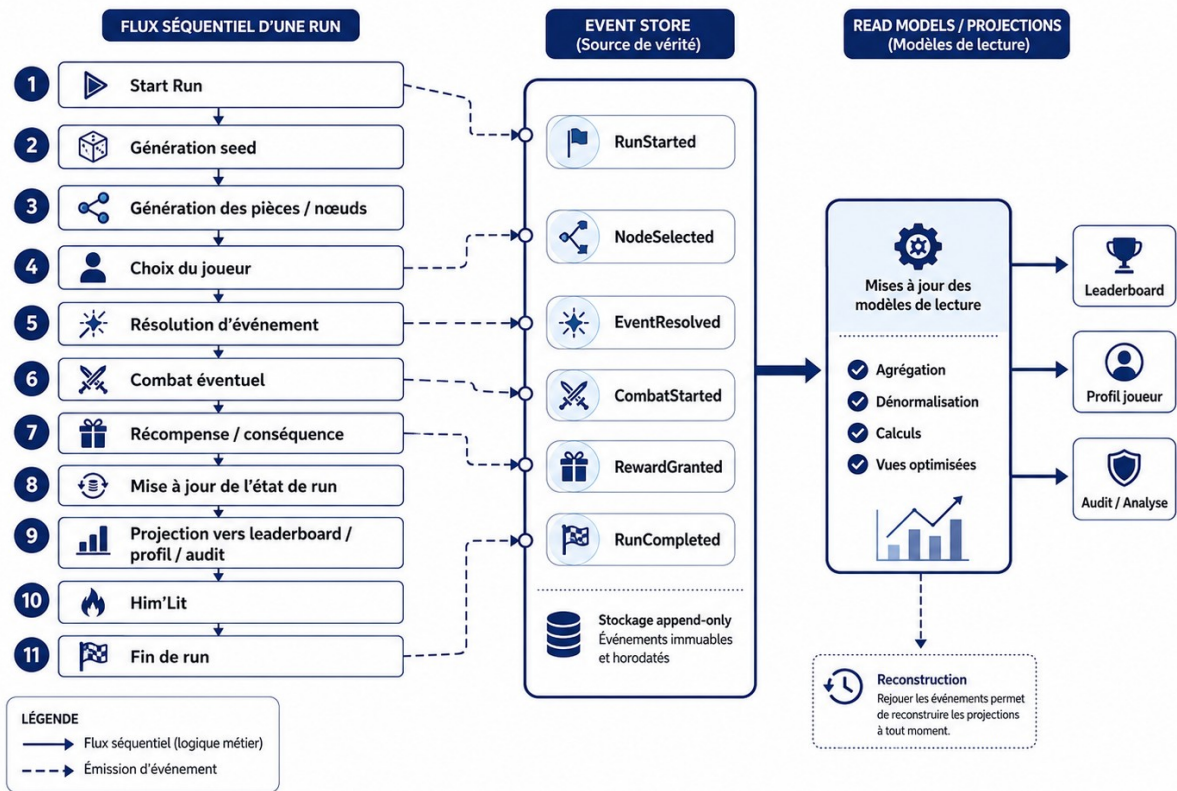


Annexe E — Sélection de salle



Annexe F — Architecture technique cible

SFD V2 — Cycle de vie d'une run et Event Sourcing



Annexe G — Cycle de vie et Event Sourcing